Foundation of Blockchain Technology (IS501)
Post-Reading Material – Lecture 16-18

Attacks on Smart Contract

Introduction to Smart Contract Security

Smart contracts are programs deployed on a blockchain that automatically execute predefined instructions when specified conditions are satisfied.

Although blockchain technology provides security through cryptography and consensus mechanisms, smart contracts can still contain programming errors and security vulnerabilities.

An attacker can exploit these vulnerabilities to:

- Steal cryptocurrency or tokens.
- Manipulate contract balances.
- Execute unauthorized transactions.
- Exploit errors in mathematical calculations.
- Cause financial losses to users.

Two important smart contract vulnerabilities discussed in this section are:

1. Reentrancy Attack
2. Integer Overflow and Underflow

2. Reentrancy Attack

2.1 Background: The DAO Attack (2016)

In 2016, a Decentralized Autonomous Organization (DAO) was established on the Ethereum blockchain.

What was DAO?

DAO was a blockchain-based investment organization whose objective was to operate as a decentralized investment body owned and managed by its members.

Important facts:

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

- It raised approximately $150 million from thousands of investors.
- It operated using smart contracts on the Ethereum blockchain.
- An attacker exploited a vulnerability in its smart contract.
- Approximately 3.6 million ETH was drained from the DAO.

What was the reason behind the attack?

The vulnerability was caused by recursive calling of an external smart contract before the original function had completed its execution.

This allowed the attacker to withdraw funds multiple times.

2.2 Why was fixing the DAO attack difficult?

Once a smart contract is deployed on a blockchain, its code generally cannot be directly modified.

Therefore, fixing the vulnerability in the deployed DAO contract was not straightforward.

The Ethereum community eventually implemented a hard fork to address the consequences of the attack.

A hard fork resulted in two separate blockchain networks:

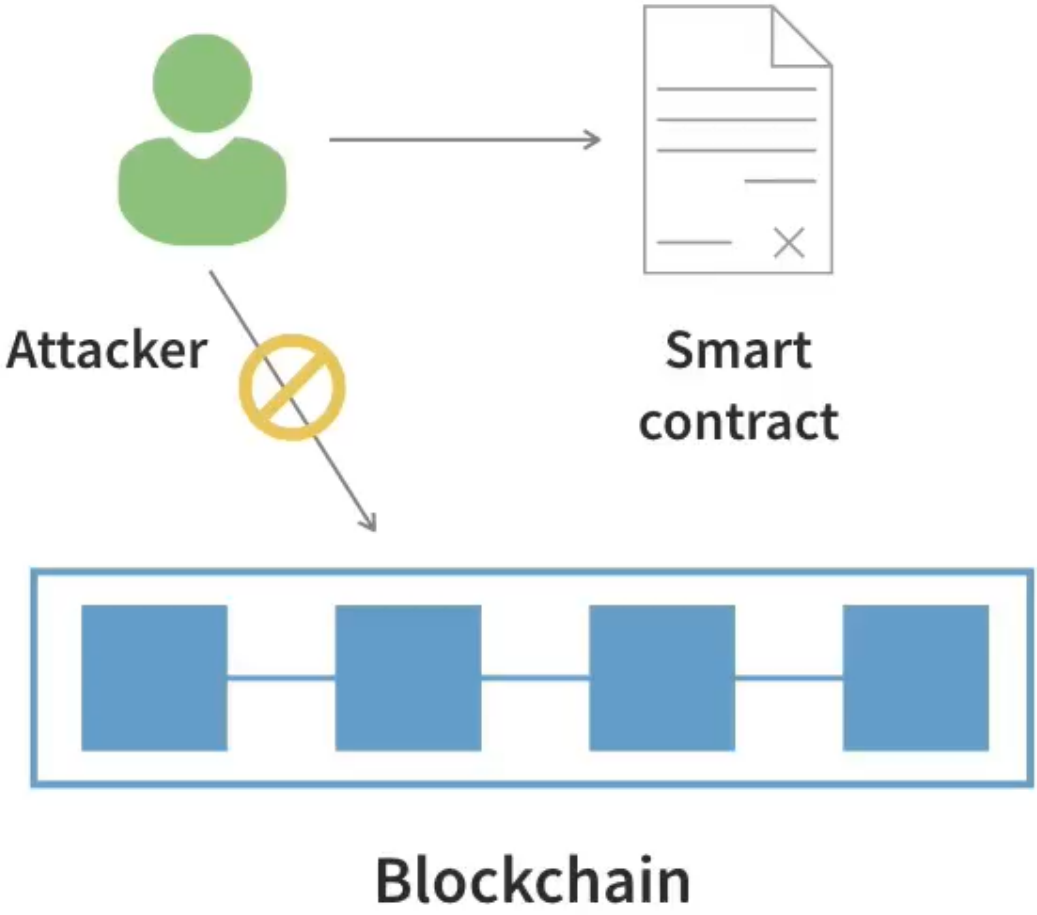
1. Ethereum (ETH): The blockchain that continued with the forked state.
2. Ethereum Classic (ETC): The original chain that continued separately.

2.3 What is a Reentrancy Attack?

Definition:

A reentrancy attack is a smart contract vulnerability in which an external contract repeatedly calls a vulnerable function before its previous execution has completed.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in



As a result, the vulnerable contract may execute the same operation multiple times using an outdated state or balance.

Simple analogy: ATM withdrawal

Imagine that a person has ₹10,000 in a bank account.

1. The person requests a withdrawal of ₹5,000.
2. The bank gives ₹5,000.
3. Before the bank updates the account balance, another withdrawal request is triggered.
4. The bank still sees the old balance of ₹10,000.
5. The person withdraws another ₹5,000.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

If the process repeats, the person may withdraw more money than the actual account balance.

This is the basic intuition behind a reentrancy attack.

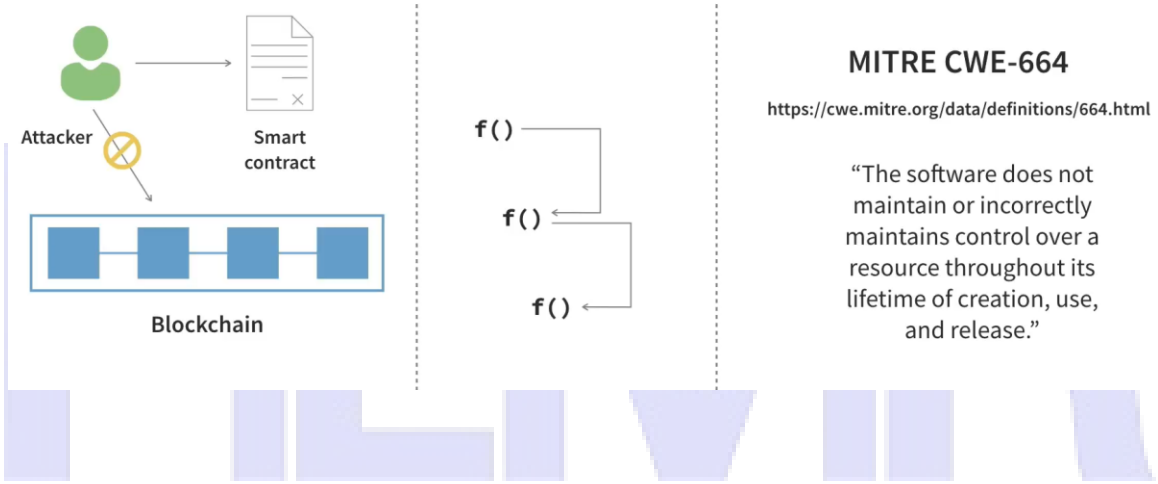
2.4 How Does a Reentrancy Attack Work?

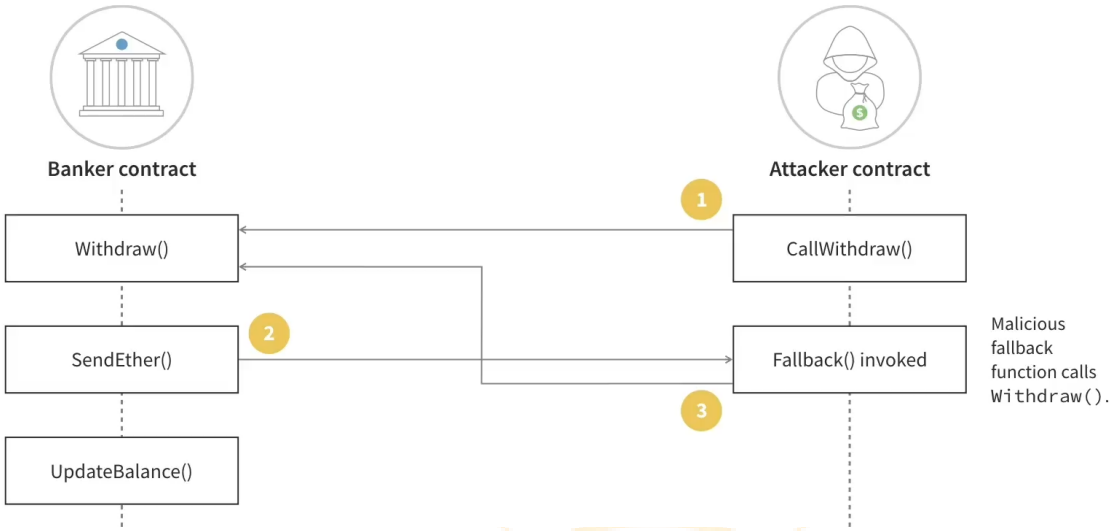
Consider two smart contracts:

- Banker Contract: Stores cryptocurrency and manages users' balances.
- Attacker Contract: A malicious contract designed to exploit the banker contract.

The banker contract provides a withdraw() function that allows users to withdraw their funds.

The attack proceeds as follows.





Step-by-step explanation

Step 1: Initial withdrawal request

The attacker contract calls the `withdraw()` function of the banker contract.

Step 2: Sending Ether

The banker contract executes the `SendEther` operation to transfer cryptocurrency to the attacker.

Step 3: Fallback function gets executed

When the attacker contract receives Ether, its `fallback()` function is triggered.

A fallback function is a special Solidity function that can execute when a contract receives Ether without a matching function call.

Step 4: Recursive call to withdraw()

Inside its fallback function, the malicious attacker contract calls the banker's `withdraw()` function again.

The important point is that the original withdrawal operation has not yet completed.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Step 5: Repeated withdrawal

The banker contract receives another withdrawal request.

Since the balance has not yet been updated, the contract may incorrectly believe that sufficient funds are still available.

The attacker can repeat this process multiple times.

Step 6: Balance update

Finally, the control returns to the banker's `UpdateBalance` operation.

However, the balance update happens too late because multiple withdrawal operations have already been initiated.

The contract transfers Ether before updating the user's balance. This allows the attacker to re-enter the withdrawal function using the old balance.

2.5 Reentrancy Attack: Solidity Code Example

Consider the following vulnerable contract: // SPDX-License-Identifier: MIT

```
pragma solidity ^0.8.21;


contract Banker {

   mapping(address => uint) public balance;


   function deposit() public payable {

      balance[msg.sender] += msg.value;

   }
```


Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

```
    function withdraw(uint amount) public {


        require(balance[msg.sender] >= amount,

                "Insufficient balance");



        // External call first

        payable(msg.sender).call{value: amount}("");



        // Balance updated later

        balance[msg.sender] -= amount;

    }
}
```

Understanding the vulnerable code:

1. require() checks whether the user has sufficient balance.
2. call() transfers Ether to the user's address.
3. The receiver's fallback function can execute.
4. The attacker can call withdraw() again.
5. The balance is reduced only after the external call.

Therefore, the attacker gets an opportunity to withdraw repeatedly before the balance is updated.

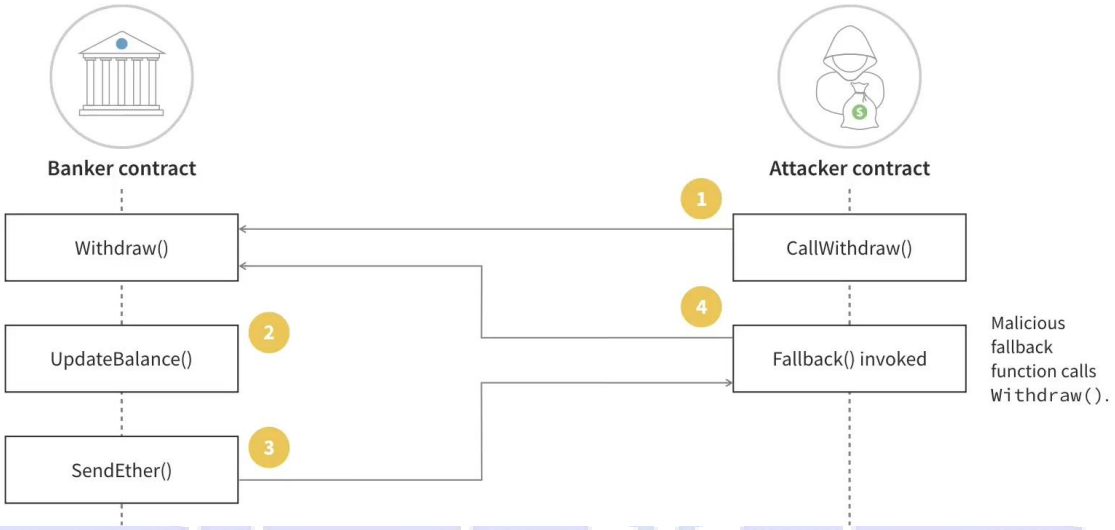
Note: This is an illustrative vulnerable example. Production code must also check the success status of external calls and protect against reentrancy.

2.6 How to Prevent a Reentrancy Attack?

The primary solution is to update the internal balance before transferring Ether to an external address.

This follows the Checks-Effects-Interactions (CEI) pattern.

- Checks: Verify the withdrawal conditions.
- Effects: Update the contract's internal state.
- Interactions: Perform external calls or transfer Ether.




```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.21;


contract Banker {

   mapping(address => uint) public balance;


   function deposit() public payable {

      balance[msg.sender] += msg.value;

   }
```

```
    function withdraw(uint amount) public {

        // 1. Checks
        require(balance[msg.sender] >= amount,
            "Insufficient balance");

        // 2. Effects
        balance[msg.sender] -= amount;

        // 3. Interactions
        (bool success, ) =
            payable(msg.sender).call{value: amount}("");

        require(success, "Transfer failed");
    }
}
```

Why does this solution work?

Suppose the user's balance is 10 ETH and the user requests 5 ETH.

1. The contract first checks that the balance is sufficient.
2. The balance is immediately reduced from 10 ETH to 5 ETH.
3. Ether is then transferred to the attacker.
4. If the attacker tries to call `withdraw()` again, the contract checks the updated balance.
5. The attacker cannot withdraw more than the remaining balance.

Prevention technique

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Follow Checks-Effects-Interactions (CEI): update the contract state before making an external call.

Additional protection: Developers can use a reentrancy guard, such as OpenZeppelin's `ReentrancyGuard`, to prevent repeated entry into protected functions.

MITRE Classification: CWE-664

The reentrancy vulnerability is associated with the broader weakness category CWE-664: Improper Control of a Resource Through its Lifetime.

According to this classification, software fails to properly control a resource during its lifecycle, including:

- Creation
- Usage
- Release

In smart contracts, incorrect management of the contract's state and resources can allow attackers to exploit the system.

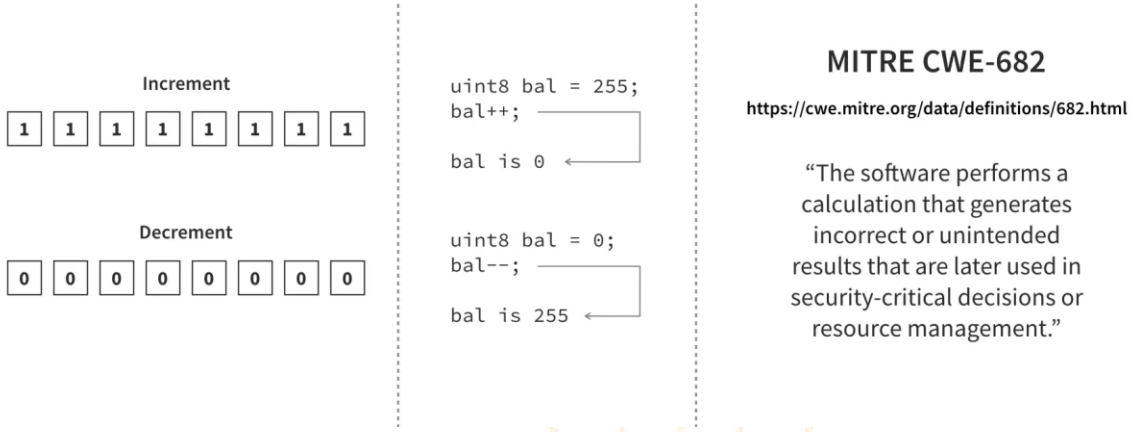
Integer Overflow and Underflow

Introduction

Integer overflow and underflow are arithmetic errors that occur when a numerical variable exceeds its permitted range.

These vulnerabilities are not limited to blockchain applications. They can occur in any software program that performs numerical calculations.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in



However, they are particularly important in smart contracts because cryptocurrency transactions involve numerical values such as:

- Token balances
- Transaction amounts
- Account balances
- Gas-related calculations
- Rewards and payments

An arithmetic error in these calculations can result in financial losses.

What is Integer Overflow?

Definition:

Integer overflow occurs when the result of an arithmetic operation exceeds the maximum value that can be stored in a particular integer variable.

Consider an unsigned 8-bit integer (uint8).

It can store values from:

0 to 255

The maximum value is 255.

What happens if we add 1 to 255?

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

255+1 = 26

But a `uint8` variable cannot store 256.

In arithmetic systems using fixed-width wrapping behaviour, the result becomes:

256 mod 256=0

Therefore:

255+1=0

This is called integer overflow.

Example:

Operation	Mathematical result	8-bit wrapping result
250 + 5	255	255
255 + 1	256	0
200 + 100	300	44

What is Integer Underflow?

Definition:

Integer underflow occurs when the result of an arithmetic operation becomes smaller than the minimum value that can be stored in an integer variable.

For an unsigned 8-bit integer:

{Minimum value}=0

Suppose we subtract 1 from 0.

0-1=-1

However, an unsigned integer cannot store a negative number.

In fixed-width wrapping arithmetic:

-1 mod 256=255

Therefore:

0-1=255

This is called integer underflow.

Operation	Mathematical result	8-bit wrapping result
10 − 5	5	5
1 − 1	0	0
0 − 1	−1	255

Understanding Overflow and Underflow in Smart Contracts

Consider a smart contract that maintains users' token balances.

Suppose a user has a balance of 100 tokens.

The contract contains a function that allows users to withdraw tokens.

If the developer fails to check the withdrawal amount properly, an attacker might attempt to withdraw more tokens than the available balance.

For example:

{Initial balance}=100

{Withdrawal amount}=150

If the contract performs an unchecked subtraction:

100-150=-50

In an 8-bit unsigned integer using wrapping arithmetic:

-50 mod 256=206

The balance may become 206 instead of correctly rejecting the transaction.

This could allow an attacker to manipulate token balances.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Important: This is an illustrative example of the danger of unchecked arithmetic. The actual result depends on the integer type, Solidity version and arithmetic checks.

Why Are Smart Contracts Vulnerable?

Integer overflow and underflow vulnerabilities become particularly dangerous when:

1. A smart contract performs arithmetic operations without proper validation.
2. Public functions allow users to modify numerical variables.
3. The contract does not check the upper and lower limits of variables.
4. The incorrect calculation is subsequently used for security-critical decisions or resource management.

For example, an incorrect token balance calculation could affect withdrawal permissions and token transfers.

MITRE Classification: CWE-682

According to MITRE, integer overflow and underflow are related to CWE-682: Incorrect Calculation.

This weakness occurs when software performs a calculation that generates incorrect or unintended results, which are subsequently used in security-critical decisions or resource management.

In simple words:

The program performs a calculation, but the calculated result is incorrect. If the program uses that incorrect result to make an important decision, it can create a security vulnerability.

How to Prevent Integer Overflow and Underflow?

There are two important mitigation techniques.

1. Use Solidity version 0.8.0 or above

Solidity 0.8.0 introduced automatic arithmetic overflow and underflow checks for ordinary arithmetic operations.

// SPDX-License-Identifier: MIT

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

```
pragma solidity ^0.8.21;


contract ArithmeticExample {


   uint8 public number = 255;


   function increment() public {
      number = number + 1;
   }
}
```

When the increment() function is executed:

255+1=256

Since 256 exceeds the maximum value of uint8, Solidity 0.8.21 automatically reverts the transaction.

The variable remains unchanged.

2. Use SafeMath Library

In older Solidity versions, such as Solidity 0.7.x and earlier, developers commonly used the SafeMath.sol library to prevent arithmetic overflow and underflow.

```
using SafeMath for uint256;


uint256 public balance = 100;
```

```
function withdraw(uint256 amount) public {

   balance = balance.sub(amount);

}
```

The SafeMath library checks whether the subtraction is valid and reverts if it would produce an underflow.

Note: SafeMath is generally unnecessary for ordinary arithmetic in Solidity 0.8.0 and above because overflow and underflow checks are built in. It is still important to avoid unchecked arithmetic blocks unless their behaviour is intentional and properly validated.

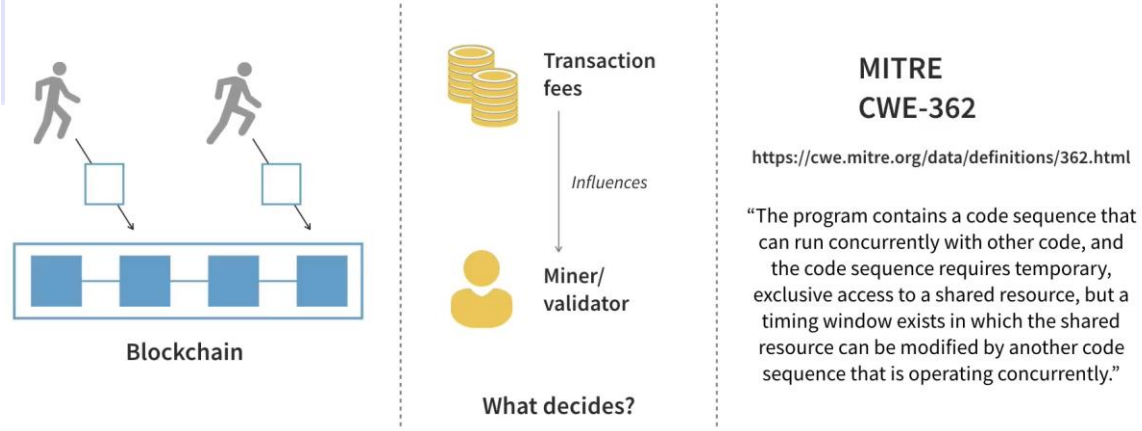
Front-Running Attack

Introduction

A front-running attack is a type of attack in which an attacker observes a pending transaction and submits their own transaction with a higher transaction fee so that it gets executed before the original transaction.

The concept of front-running originally comes from traditional financial markets, where a trader uses advance knowledge of another person's transaction to gain a financial advantage.

In blockchain networks, attackers exploit the visibility of pending transactions to manipulate transaction ordering.



Understanding Mempool

Before a transaction is recorded in a blockchain block, it is generally stored in a temporary waiting area called the Mempool (Memory Pool).

Definition:

A mempool is a collection of unconfirmed transactions waiting to be selected by block producers and included in a blockchain block.

When a user submits a transaction:

1. The transaction is broadcast to the blockchain network.
2. It enters the mempool.
3. Block producers select transactions from the mempool.
4. Selected transactions are included in a new block.
5. The block is added to the blockchain after the required consensus process.

Important point: In publicly observable mempools, pending transaction information may be visible to other network participants before the transaction is confirmed.

This visibility creates an opportunity for attackers.

How Are Transactions Selected from the Mempool?

Block producers have an incentive to select transactions that offer higher transaction fees.

Generally:

- Higher transaction fee → Greater incentive for the block producer.
- Lower transaction fee → Comparatively lower incentive.

Therefore, an attacker can exploit transaction ordering by submitting a transaction with a higher fee than the original pending transaction.

Note: Transaction ordering is not determined exclusively by fees. Actual selection depends on the blockchain protocol, block producer policies, network conditions and transaction type.

Working of a Front-Running Attack

Consider the following example.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Suppose Alice wants to purchase a token at a particular price using a decentralized exchange (DEX).

The transaction is submitted to the blockchain network and becomes visible in the mempool.

An attacker, Bob, observes Alice's pending transaction and realizes that the transaction may increase the token price.

Bob submits his own transaction with a higher transaction fee so that his transaction gets executed before Alice's transaction.

Step-by-step explanation

Step 1: Alice submits a transaction

Alice submits a transaction to purchase 100 tokens through a decentralized exchange.

Step 2: Transaction enters the mempool

Alice's transaction is waiting for confirmation and is visible in the public mempool.

Step 3: Attacker observes the transaction

Bob monitors the mempool and identifies Alice's pending transaction.

Step 4: Attacker submits a higher-fee transaction

Bob submits his own transaction to purchase the same token with a higher transaction fee.

Step 5: Attacker's transaction gets executed first

The block producer selects Bob's transaction before Alice's transaction.

Step 6: Alice's transaction gets executed

After Bob's transaction changes the market conditions or token price, Alice's transaction is executed.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

As a result, Alice may receive fewer tokens or experience a less favourable price.

An attacker uses information about a pending transaction to gain an advantage by influencing the order in which transactions are executed.

Another Simple Example: Queue at a Ticket Counter

Imagine that Alice is standing in a queue to purchase a movie ticket.

Bob observes that Alice is about to purchase the last available ticket.

Instead of waiting in the queue, Bob pays an additional amount to get priority and purchases the ticket before Alice.

Now Alice cannot purchase the ticket.

Similarly, in a front-running attack, an attacker uses transaction information and transaction ordering to gain an advantage over another user.

MITRE Classification: CWE-362

Front-running is associated with the broader weakness category:

CWE-362: Concurrent Execution using Shared Resource with Improper Synchronization (Race Condition).

According to MITRE, this weakness occurs when two or more operations access a shared resource concurrently, and an operation can modify that resource during a timing window.

In simple words:

Two transactions compete to access or modify the same resource. The transaction that executes first may change the conditions for the transaction that executes later.

In blockchain, this can happen when multiple pending transactions interact with the same smart contract state.

How to Prevent Front-Running Attacks?

Developers can use a combination of blockchain platform-level and smart contract-level security techniques.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

1. Remove the benefit of transaction ordering

Smart contracts should be designed so that an attacker cannot gain an unfair advantage simply by executing their transaction before another user's transaction.

For example, an application can use batch processing or order-independent settlement where appropriate.

2. Use a Commit-Reveal Scheme

A commit-reveal scheme is a security technique in which users first submit a commitment to their intended transaction and reveal the actual details later.

It consists of two phases:

- Commit Phase
- Reveal Phase

Commit Phase:

1. The user decides the transaction details.
2. A cryptographic hash of the transaction details and a secret value (nonce) is generated.
3. Only the hash is submitted to the blockchain.
4. Other users cannot directly identify the committed transaction details from the hash.

Reveal Phase:

1. After the commitment period ends, the user reveals the original transaction details and secret value.
2. The smart contract calculates the hash again.
3. It compares the newly calculated hash with the previously submitted commitment.
4. If both hashes match, the commitment is accepted.

Example:

Alice wants to bid ₹10,000 in an auction.

Instead of immediately revealing her bid, she submits a cryptographic commitment.

Later, she reveals her actual bid of ₹10,000.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

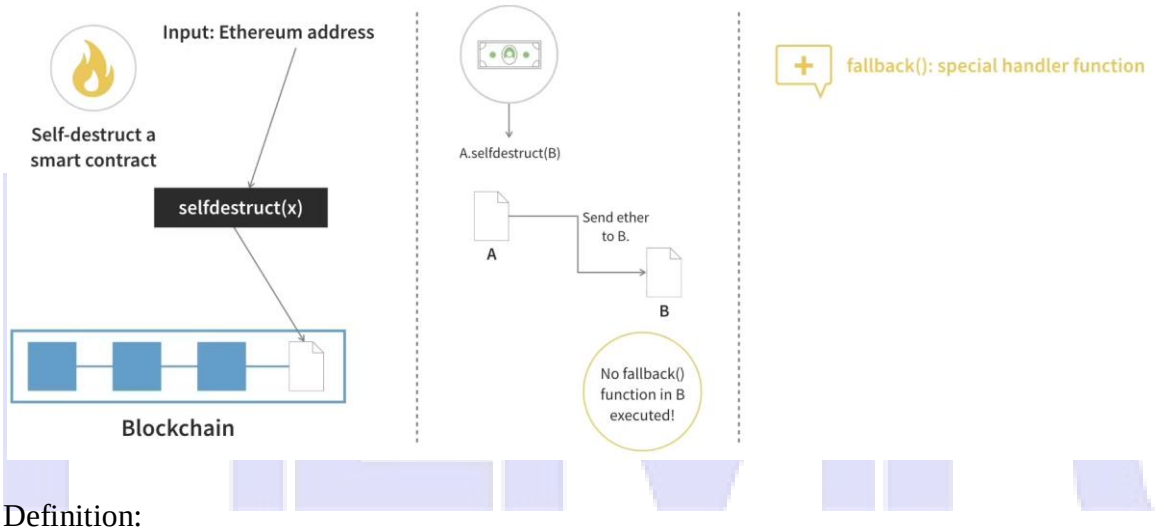
This prevents other participants from directly observing her bid during the commitment phase.

Important: Commit-reveal reduces information leakage, but it must be carefully designed to prevent problems such as invalid reveals, non-reveals and timing manipulation. It is not a universal solution to every type of front-running.

Self-Destruct Related Vulnerability (Forced Ether Transfer)

Introduction to Self-Destruct in Solidity

Solidity provides a special operation called `selfdestruct()` that historically allowed developers to terminate a smart contract and transfer its remaining Ether balance to a specified address.



Definition:

The self-destruct operation is used to terminate a contract's execution and transfer its Ether balance to a specified Ethereum address.

For example:

selfdestruct(payable(recipient));

Here:

- `selfdestruct()` performs the termination operation.
- `recipient` is the Ethereum address that receives the Ether.
- The remaining Ether balance of the contract is transferred to the recipient.

Important update regarding Solidity: Since the Dencun upgrade (March 2024), Ethereum's EIP-6780 changed the behaviour of `SELFDESTRUCT`. For contracts not created and destroyed in the same transaction, it generally transfers Ether but does not actually remove the contract's code and storage. Therefore, older explanations that say self-destruct always permanently deletes a deployed contract are outdated.

What is the Security Problem?

The security issue arises because Ether can be transferred to a smart contract without executing its `receive()` or `fallback()` function.

Normally, when a contract receives Ether through a regular transfer or call, its receiving function is executed.

The receiving function can contain validation logic to decide whether the contract should accept the payment.

However, when Ether is forcibly transferred through `SELFDESTRUCT`, the receiving contract's fallback or receive function is not executed.

This means that the receiving contract cannot validate the incoming transfer.

Main vulnerability

A smart contract may receive Ether without executing its fallback or receive function, bypassing the validation logic implemented inside those functions.

Understanding the Fallback Function

Before understanding the attack, we need to understand the fallback function in Solidity.

Definition:

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

A fallback function is a special function in Solidity that is executed when a contract receives a call that does not match any existing function, or when Ether is sent without calldata and no suitable `receive()` function is available.

Syntax:

```
fallback() external payable {
    // Logic to handle incoming calls
}
```

The `payable` keyword allows the function to receive Ether.

Example:

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.21;


contract Receiver {

  uint public receivedAmount;


  fallback() external payable {

    receivedAmount += msg.value;

  }
}
```

When the contract receives Ether through a call that triggers its fallback function, the function executes and updates `receivedAmount`.

Important distinction:

Solidity provides two special functions for handling incoming Ether:

Function	Purpose

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

`receive()`	Executes when Ether is received with empty calldata
`fallback()`	Executes when no matching function is found, or when empty calldata is received and no `receive()` function exists

Both functions must be marked `payable` to accept Ether.

How Does the Forced Ether Transfer Attack Work?

Consider two smart contracts:

- Receiver Contract: A smart contract that expects to receive Ether only through its approved payment mechanism.
- Attacker Contract: A malicious contract that forcibly sends Ether to the receiver contract.

The receiver contract contains a fallback function that validates incoming payments.

However, the attacker exploits the self-destruct mechanism to bypass this validation.

Step-by-step explanation

Step 1: Receiver contract is deployed

A receiver contract is deployed on the Ethereum blockchain.

It contains a fallback or receive function that checks whether incoming Ether is appropriate.

Step 2: Attacker creates another contract

The attacker deploys a separate smart contract containing Ether.

Step 3: Attacker invokes self-destruct

The attacker executes the self-destruct operation and specifies the receiver contract's address as the recipient.

Step 4: Ether is forcibly transferred

The Ether balance of the attacker contract is transferred to the receiver contract.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Step 5: Fallback function is bypassed

The receiver contract's fallback or receive function is not executed.

Therefore, its validation logic is bypassed.

Step 6: Receiver contract gets unexpected Ether

The receiver contract now has an Ether balance that it did not receive through its expected payment mechanism.

This can create problems if the receiver contract relies on its actual Ether balance for important calculations or security decisions.

Why Is This Attack Dangerous?

Consider a smart contract that checks whether it has received the expected amount of Ether using:

require(address(this).balance == expectedAmount);

Suppose the contract expects 10 ETH.

Initially:

Contract Balance=10 ETH

An attacker forcibly sends another 5 ETH.

Now:

Contract Balance=15 ETH

But the contract's internal payment records may still indicate that only 10 ETH has been received through its approved payment mechanism.

Therefore, the actual blockchain balance and the contract's internal accounting may become inconsistent.

This can cause problems in payment validation, accounting and other business logic.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

How to Prevent Forced Ether Transfer Vulnerabilities?

The primary recommendation is:

Do not rely exclusively on `address(this).balance` to validate incoming payments or maintain application accounting.

Instead, maintain a separate internal variable to track payments received through the contract's approved functions.

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.21;


contract Receiver {


   uint public trackedBalance;


   function deposit() public payable {


      require(msg.value > 0,

          "Invalid payment");


      trackedBalance += msg.value;

   }

}
```

Understanding the solution:

- `address(this).balance` represents the actual Ether balance held by the contract.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

- `trackedBalance` represents the Ether deposited through the approved `deposit()` function.

Suppose:

Description	Amount
Ether received through deposit()	10 ETH
Ether forcibly transferred	5 ETH
Actual contract balance	15 ETH
Internal tracked balance	10 ETH

The internal accounting remains 10 ETH even though the actual balance is 15 ETH.

Therefore, the application can distinguish between recorded deposits and unexpected Ether transfers.

Important: The internal variable should be treated as an accounting record, not as proof of every possible Ether transfer. The contract should also be designed to handle unexpected surplus Ether safely.

Smart Contract Attack Registry

Smart contract security is continuously evolving. New vulnerabilities and attack techniques may emerge as blockchain platforms and decentralized applications become more complex.

Therefore, developers should regularly study known attacks and security vulnerabilities.

One useful resource is the Smart Contract Attack Registry.

It provides information about known smart contract attacks and security incidents.

Developers can use it to:

- Study previously exploited vulnerabilities.
- Understand how attackers exploit smart contracts.
- Identify common coding mistakes.
- Learn prevention techniques.
- Improve smart contract security practices.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Secure Smart Contract Design Practices

The instructor concludes by emphasizing that developers should follow secure design practices to prevent avoidable vulnerabilities and make smart contracts robust against previously unknown attacks, commonly called zero-day attacks.

Some important practices include:

1. Secure coding: Follow established Solidity coding standards and avoid unsafe programming patterns.
2. Input validation: Validate user inputs before executing critical operations.
3. Access control: Restrict sensitive functions to authorized users.
4. Reentrancy protection: Follow Checks-Effects-Interactions and use reentrancy guards where appropriate.
5. Safe arithmetic: Use Solidity 0.8 or later and avoid unchecked arithmetic unless necessary.
6. Transaction-ordering protection: Use commit-reveal schemes or suitable transaction-order-independent designs.
7. Secure Ether handling: Never assume Ether can only enter a contract through its fallback or receive function.
8. Testing: Test smart contracts against unexpected inputs and malicious interactions.
9. Security auditing: Conduct code reviews and independent security audits before deployment.
10. Continuous monitoring: Monitor deployed contracts for suspicious transactions and unusual behaviour.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in